MAJOR PROJECT REPORT

Fraud Detection Analysis and Risk Interpretation

Submitted as part of the Data Science Internship

Submitted by

Danishveer Singh

Internship Provider

Skillairo Edutech Pvt. Ltd.

Domain: Data Science

October 2026

Table of Contents

1. Abstract **3**

2. Introduction **3**

2.1 Problem Statement 3

2.2 Objectives 3

3. Dataset Description **3**

4. Tools and Technologies Used **3**

5. Methodology **4**

6. Implementation and Results **5**

6.1 Data Preparation and SMOTE 5

6.2 Logistic Regression 6

6.3 Random Forest 7

6.4 Comparison of the Models 7

6.5 Risk Level Interpretation 8

6.6 Feature Importance 9

7. Interpretation for Risk Teams **10**

8. Limitations and Future Work **11**

9. Conclusion **11**

10. Links **11**

1. Abstract

This report is about my major project, *Fraud Detection Analysis and Risk Interpretation*, done during my Data Science internship at Skillairo. Using the Credit Card Fraud dataset, I built two models, Logistic Regression and Random Forest, to detect fraudulent transactions. Since only 0.173% of the transactions are fraud, I used SMOTE on the training data to handle the imbalance and evaluated the models using precision, recall, ROC-AUC and PR-AUC. The Random Forest gave the better result (precision 0.81, recall 0.80, PR-AUC 0.87). I also divided the transactions into High, Medium and Low risk levels and found the most important features, so that the results can be understood by a risk team.

2. Introduction

2.1 Problem Statement

Fraudulent activities are difficult to detect in large transaction datasets because they form a very small part of the data.

2.2 Objectives

- To analyze the transaction data and identify fraud patterns and risks.
- To handle the imbalanced data properly.
- To build fraud detection models and compare them.
- To interpret the results for risk teams.

3. Dataset Description

I used the Credit Card Fraud Detection dataset from Kaggle (mlg-ulb/creditcardfraud). It has 284,807 transactions made by European cardholders in two days. There are 28 anonymized features (V1 to V28), the Time and Amount columns, and the target column Class (0 = Normal, 1 = Fraud). Out of all transactions, 284,315 are normal and only 492 are fraud (0.173%). There are no missing values.

4. Tools and Technologies Used

- **Python 3** with **Pandas** and **NumPy** for handling the data.
- **Scikit-learn** for scaling, splitting the data, the models and the evaluation metrics.
- **Imbalanced-learn (SMOTE)** to balance the training data.
- **Matplotlib** for graphs and **JupyterLab** as the working environment.

5. Methodology

My work was done in the following steps:

- **Scaling:** I standardized the Amount and Time columns using StandardScaler and removed the original columns. The V columns were already scaled.
- **Train-test split:** I split the data into 80% training and 20% testing using stratify so that both parts have the same fraud ratio. The training set has 227,451 normal and 394 fraud transactions. The test set has 56,864 normal and 98 fraud transactions.
- **Handling imbalance:** I applied SMOTE on the training data only, which made both classes 227,451 each. I did not touch the test set so that the results show the real situation.
- **Model building:** I trained Logistic Regression as the baseline model and Random Forest (200 trees) as the second model.
- **Evaluation:** I used the confusion matrix, precision, recall, F1-score, ROC-AUC and PR-AUC. Accuracy is not useful here because 99.8% of the transactions are normal.
- **Risk interpretation:** I converted the fraud probabilities of the Random Forest into risk levels and checked the feature importance.

6. Implementation and Results

6.1 Data Preparation and SMOTE

I loaded the data and confirmed the class counts. After scaling and splitting, I applied SMOTE to the training set. This created synthetic fraud samples so that the model can learn fraud patterns better.

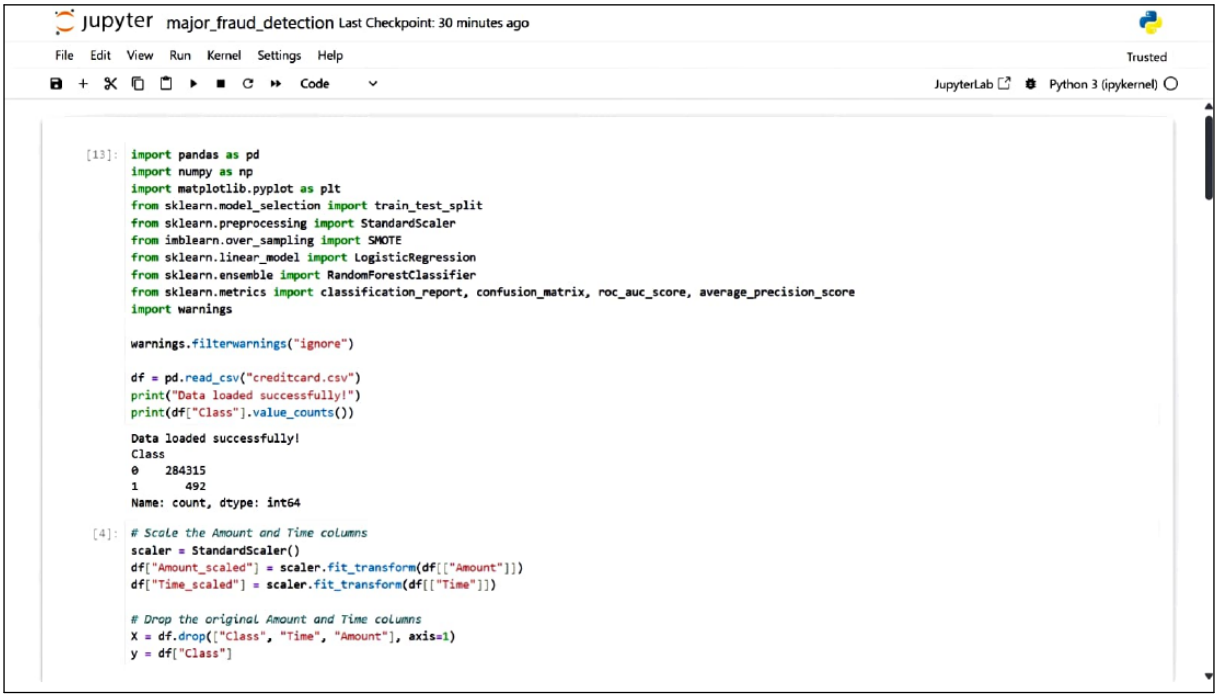


Fig. 1: Imports, loading the data and class counts



Fig. 2: Scaling, train-test split and training class counts

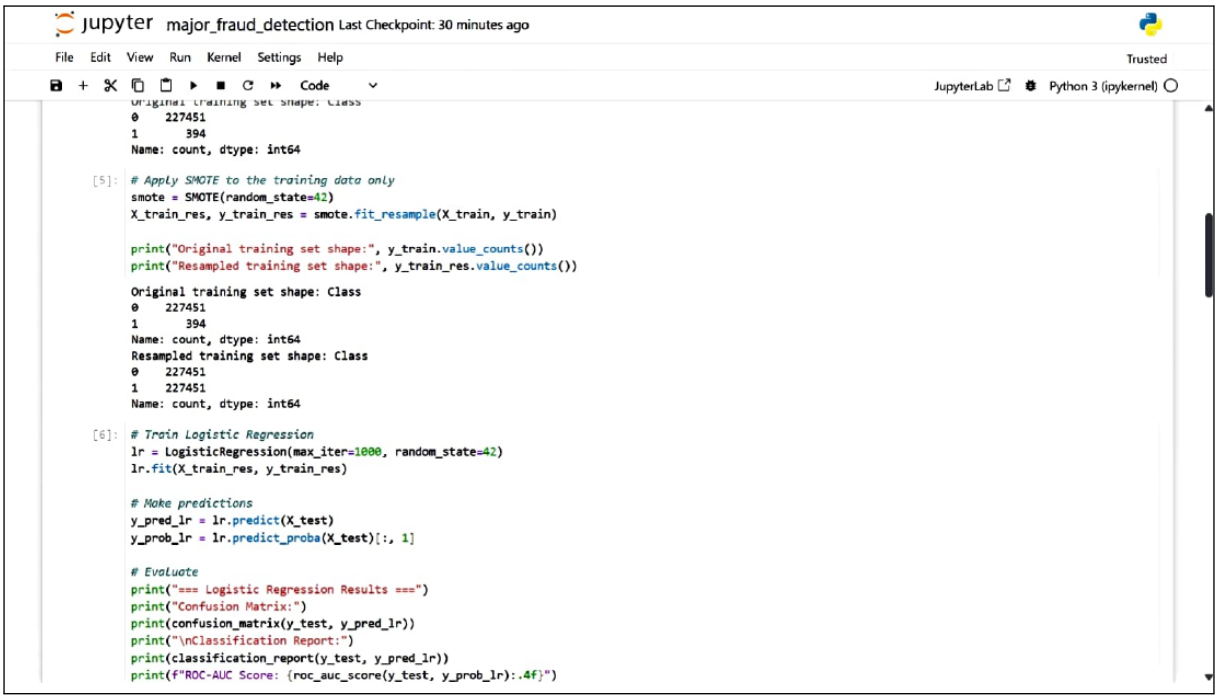


Fig. 3: Training data after applying SMOTE

6.2 Logistic Regression

The Logistic Regression model found 90 of the 98 fraud cases (recall 0.92). But it also marked 1,458 normal transactions as fraud, so its precision is only 0.06 and the F1-score is 0.11. The ROC-AUC is 0.9698 and the PR-AUC is 0.7249.

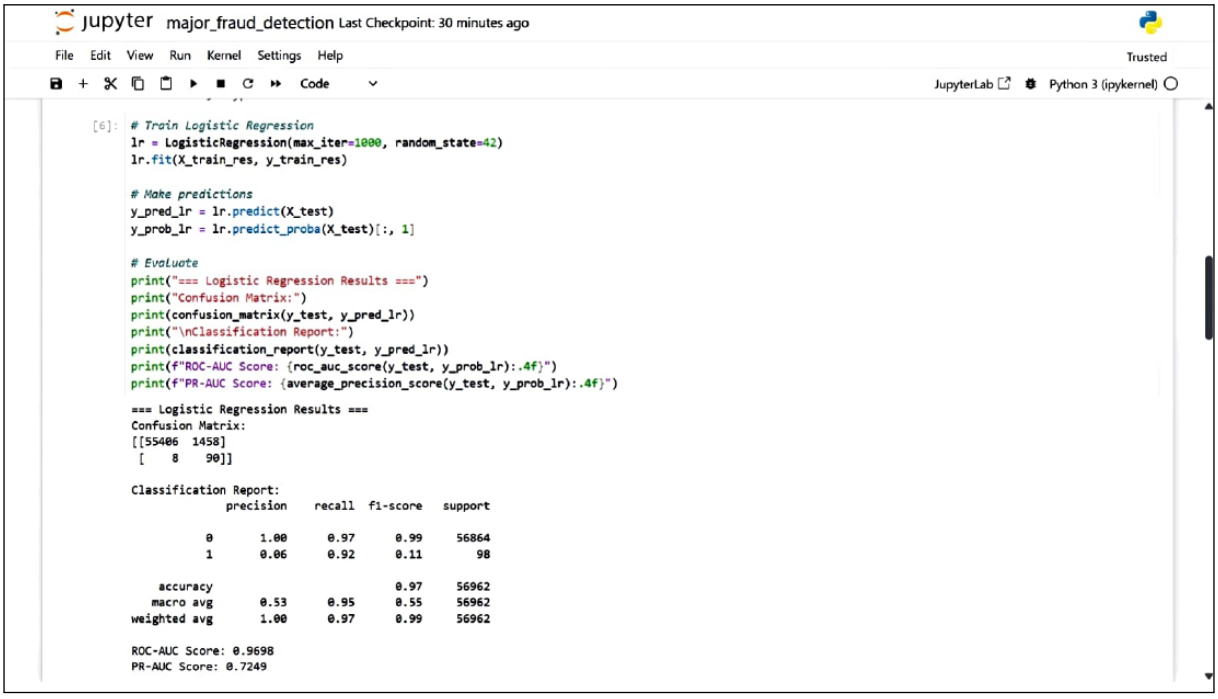


Fig. 4: Results of Logistic Regression

6.3 Random Forest

The Random Forest model found 78 of the 98 fraud cases (recall 0.80) with only 18 false alarms. Its precision is 0.81 and the F1-score is 0.80. The ROC-AUC is 0.9716 and the PR-AUC is 0.8699.

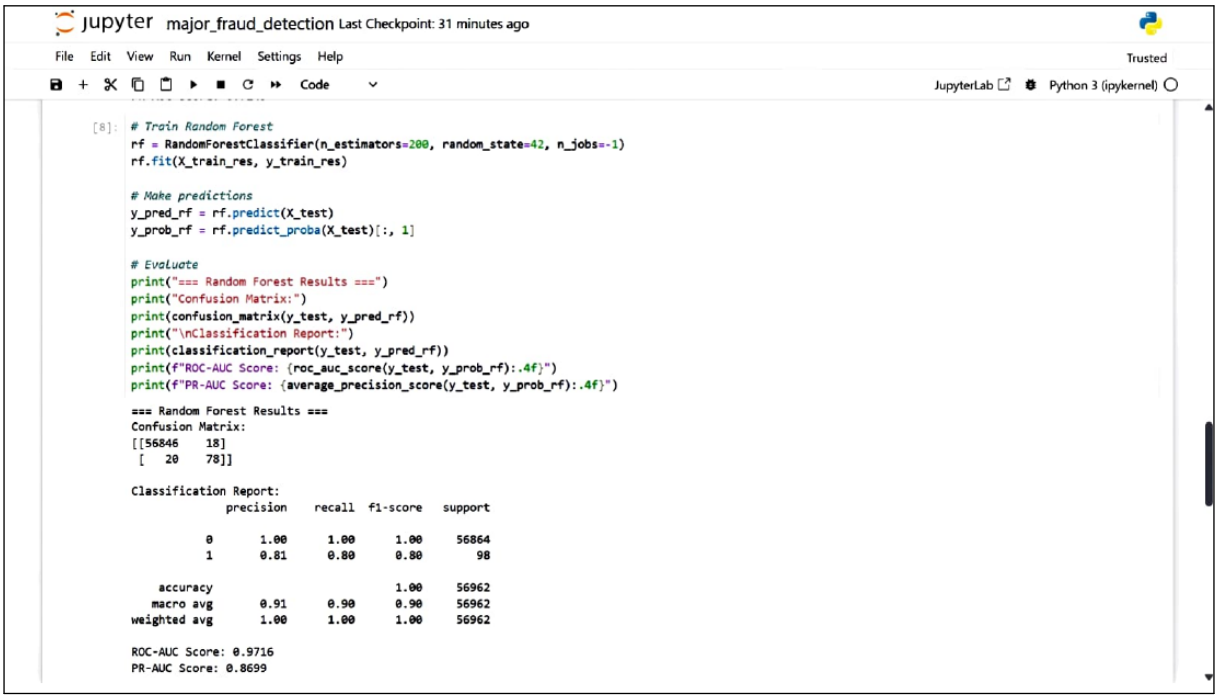


Fig. 5: Results of Random Forest

6.4 Comparison of the Models

Measure	Logistic Regression	Random Forest
Frauds caught (true positives)	90	78
Frauds missed (false negatives)	8	20
False alarms (false positives)	1,458	18
Normal correctly found (true negatives)	55,406	56,846
Precision (fraud)	0.06	0.81
Recall (fraud)	0.92	0.80
F1-score (fraud)	0.11	0.80
ROC-AUC	0.9698	0.9716
PR-AUC	0.7249	0.8699

Table 1: Comparison of Logistic Regression and Random Forest on the test set

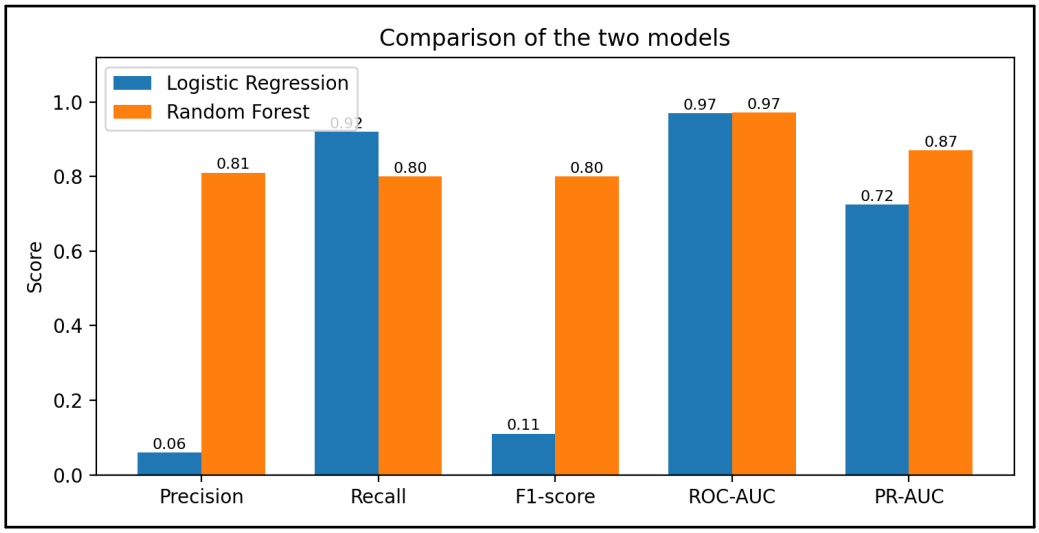


Fig. 6: Graph comparing the two models

I selected the Random Forest as the better model. The ROC-AUC of both models is almost the same, but the PR-AUC is much higher for Random Forest (0.8699 against 0.7249), and PR-AUC is a better measure when the data is highly imbalanced. With Logistic Regression an analyst would have to check about 17 false alarms for every fraud found (1,458 / 90), which is not practical. With Random Forest it is about 1 false alarm for every 4 frauds found (18 / 78). The disadvantage is that Random Forest misses 12 more frauds than Logistic Regression.

6.5 Risk Level Interpretation

To make the output useful for a risk team, I converted the fraud probability given by the Random Forest into three risk levels: High Risk (probability 0.8 or more), Medium Risk (0.4 to below 0.8) and Low Risk (below 0.4).



Fig. 7: Code for risk levels and feature importance

Risk Level	Rule	Transactions	Suggested action
High Risk	Probability 0.8 or more	74	Hold the transaction and send it for immediate review
Medium Risk	0.4 to below 0.8	31	Ask for extra verification or add to manual review
Low Risk	Below 0.4	56,857	Approve and continue monitoring

Table 2: Risk levels of the 56,962 test transactions

Only 105 of the 56,962 transactions (about 0.18%) need to be checked by a person, which is a small workload for a risk team.

6.6 Feature Importance

According to the Random Forest, the most important feature is V14 (0.200), followed by V10 (0.121), V4 (0.116), V17 (0.093), V12 (0.079) and V11 (0.073). Many of these (V14, V17, V12 and V10) were also strongly correlated with fraud in my minor project, so both analyses support each other.

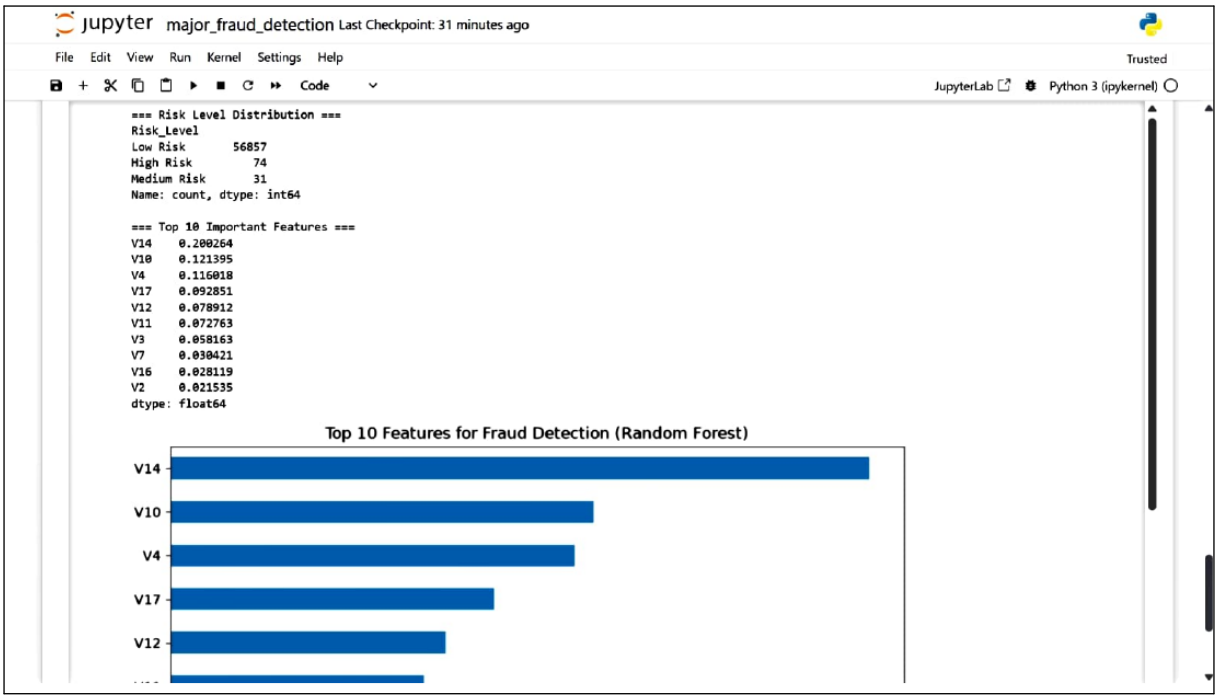


Fig. 8: Risk level count and top 10 feature importance

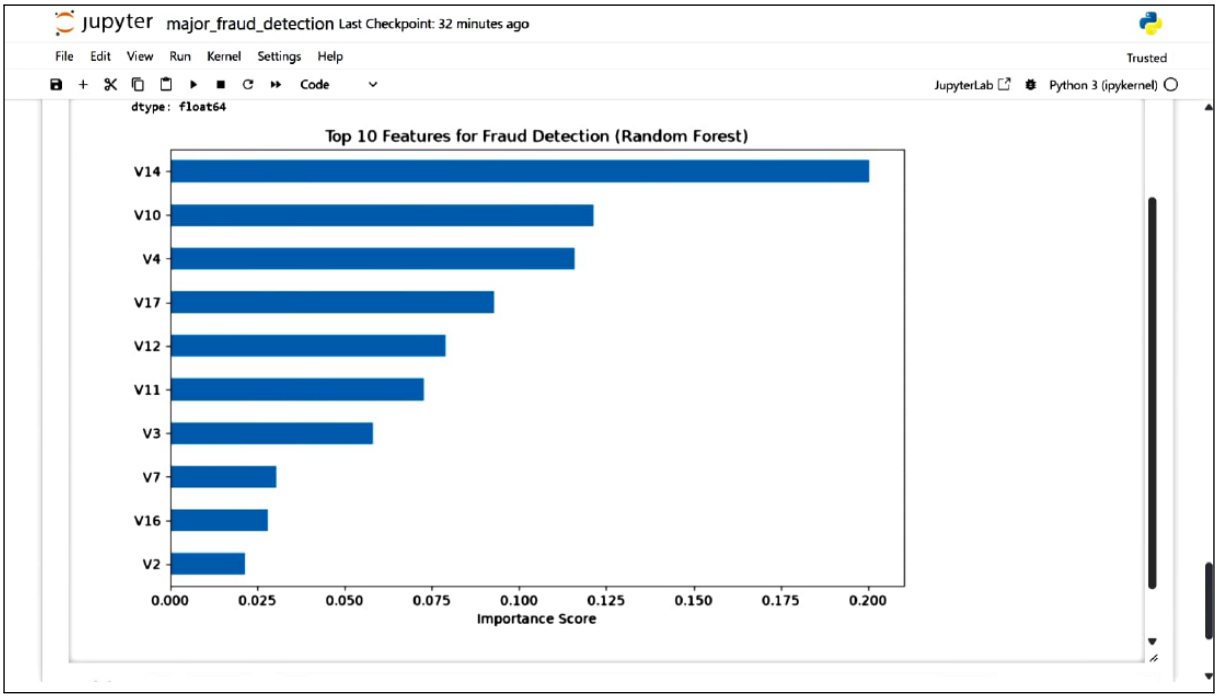


Fig. 9: Top 10 features for fraud detection (Random Forest)

7. Interpretation for Risk Teams

- The Random Forest should be used as the main model, because it finds about 80% of the fraud and 81% of its alerts are correct.
- The transactions should be handled according to risk level: High Risk first, then Medium Risk, while Low Risk can be approved.

- The thresholds of 0.4 and 0.8 can be changed. If missing a fraud is more costly than checking extra alerts, the thresholds can be lowered.
- The features V14, V10, V4, V17, V12 and V11 should be monitored because they influence the fraud score the most.
- Accuracy should not be used to judge the models, because both models have 97% or higher accuracy but Logistic Regression has only 6% precision.

8. Limitations and Future Work

- I fitted the scaler on the whole dataset before splitting it, which can cause a small data leakage. It is better to fit it only on the training data.
- I used only one train-test split and the test set has only 98 fraud cases, so the scores may change with a different split. Cross-validation can be used to get a more stable result.
- I did not tune the hyperparameters and I used the default threshold of 0.5 for the classification reports.
- The risk thresholds were chosen by me and were not optimized. I also did not check how many real frauds fall in each risk level.
- The features are anonymized, so I could not explain them in business terms. In future, models like XGBoost or LightGBM and SHAP values can be tried.

9. Conclusion

In this project I built a fraud detection workflow using a stratified split, SMOTE on the training data, two models and proper evaluation metrics. The Random Forest performed much better than the Logistic Regression baseline for practical use (PR-AUC 0.8699 against 0.7249 and precision 0.81 against 0.06) and detected 78 of the 98 frauds in the test set. By dividing the transactions into High, Medium and Low risk and finding the important features, I made the results easier for a risk team to use.

10. Links

GitHub link: https://github.com/danishveersigh/Credit-Card-Fraud-Analysis

Dataset: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud